

DSA0306 -Natural Language Processing

Vimala K

192424276

Program 1:

Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

Coding:

```
import re

text = "Anil and Arun are learning Artificial Intelligence."

match = re.search(r"\bArun\b", text)

if match:

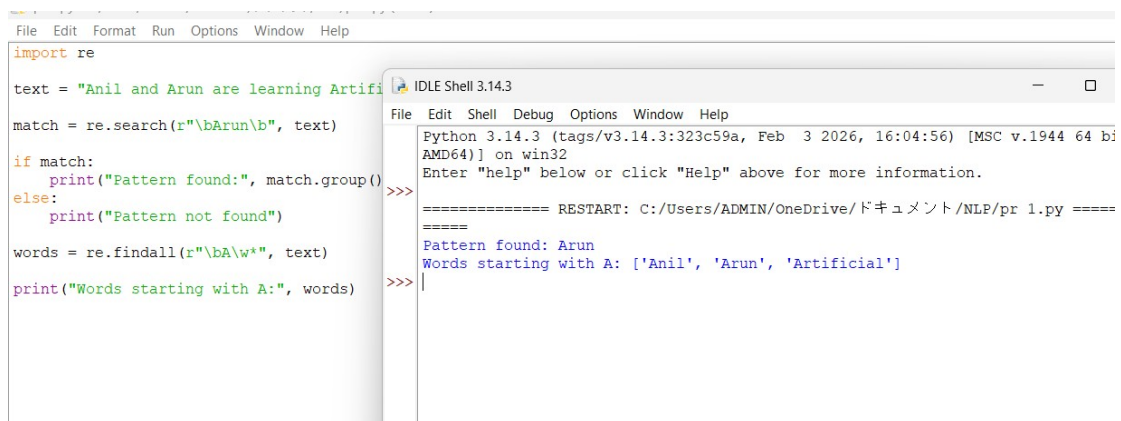
    print("Pattern found:", match.group())

else:

    print("Pattern not found")

words = re.findall(r"\bA\w*", text)

print("Words starting with A:", words)
```

A screenshot of a Python IDE window. The left pane shows the code from the previous block. The right pane, titled 'IDLE Shell 3.14.3', shows the execution output. It starts with a restart message, then displays 'Pattern found: Arun' and 'Words starting with A: ['Anil', 'Arun', 'Artificial']'.

```
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bi
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/pr 1.py =====
Pattern found: Arun
Words starting with A: ['Anil', 'Arun', 'Artificial']
>>>
```

Program 2:

Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

Coding:

```
def accepts_ending_with_ab(string):

    if string.endswith("ab"):
```

```

        return True

    else:

        return False

strings = ["ab", "cab", "aab", "abc", "helloab", "aba"]

for string in strings:

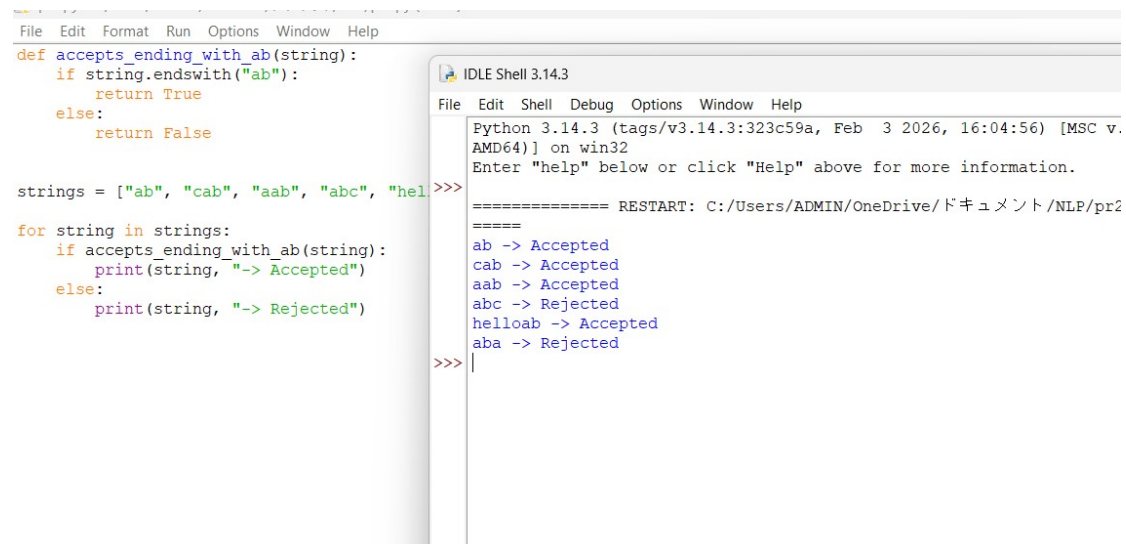
    if accepts_ending_with_ab(string):

        print(string, "-> Accepted")

    else:

        print(string, "-> Rejected")

```



```

File Edit Format Run Options Window Help
def accepts_ending_with_ab(string):
    if string.endswith("ab"):
        return True
    else:
        return False

strings = ["ab", "cab", "aab", "abc", "hel"]>>>
for string in strings:
    if accepts_ending_with_ab(string):
        print(string, "-> Accepted")
    else:
        print(string, "-> Rejected")

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.
AMD64] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/pr2
=====
ab -> Accepted
cab -> Accepted
aab -> Accepted
abc -> Rejected
helloab -> Accepted
aba -> Rejected
>>>

```

Program 3

Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

Coding:

```

import nltk

from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

nltk.download('omw-1.4')

lemmatizer = WordNetLemmatizer()

text = "Students are studying Python and birds are flying."

words = text.replace(".", "").split()

print("Word\t\tLemma")

```

```
print("-----")
```

```
for word in words:
```

```
    print(f"{word:12}{lemmatizer.lemmatize(word)}")rd))
```

```

par 3.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/par 3.py (3.14.3)
File Edit Format Run Options Window Help
import nltk
from nltk.stem import WordNetLemmatizer

ltk.download('wordnet')
ltk.download('omw-1.4')

lemmatizer = WordNetLemmatizer()

ext = "Students are studying Python and birds are flying."
ords = text.replace(".", "").split()

rint("Word\t\tLemma")
rint("-----")

or word in words:
    print(f"{word:12}{lemmatizer.lemmatize(word)}")

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/par 3.py =====
=====
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\ADMIN\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data] C:\Users\ADMIN\AppData\Roaming\nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
Word      Lemma
-----
Students  Students
are        are
studying  studying
Python    Python
and        and
birds     bird
are        are
flying    flying
>>>

```

Program 4:

Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

Coding:

```

def generate_plural(noun):
    if noun.endswith(("s", "x", "z", "ch", "sh")):
        plural = noun + "es"
    elif noun.endswith("y") and noun[-2].lower() not in "aeiou":
        plural = noun[:-1] + "ies"
    else:
        plural = noun + "s"

    return plural

```

```
words = ["cat", "bus", "box", "baby", "toy", "book"]
```

```

print("Singular\tPlural")
for word in words:
    print(f"{word:10}\t{generate_plural(word)}")

```

```

File Edit Format Run Options Window Help
def generate_plural(noun):
    if noun.endswith(("s", "x", "z", "ch", "sh")):
        plural = noun + "es"
    elif noun.endswith("y") and noun[-2].lower() not in "aeiou":
        plural = noun[:-1] + "ies"
    else:
        plural = noun + "s"
    return plural

words = ["cat", "bus", "box", "baby", "toy", "book"]
print("Singular\tPlural")
print("-----")

for word in words:
    print(f"{word:10}\t{generate_plural(word)}")

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/
=====
Singular      Plural
-----
cat           cats
bus           buses
box           boxes
baby          babies
toy           toys
book          books
>>>

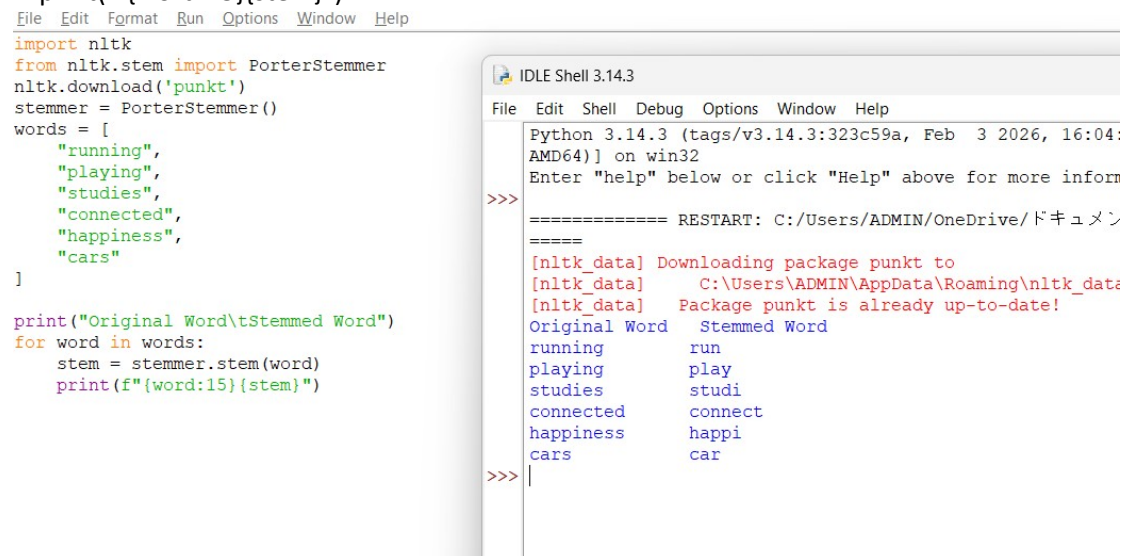
```

Program 5:

Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

Coding:

```
import nltk
from nltk.stem import PorterStemmer
nltk.download('punkt')
stemmer = PorterStemmer()
words = [
    "running",
    "playing",
    "studies",
    "connected",
    "happiness",
    "cars"
]
print("Original Word\tStemmed Word")
for word in words:
    stem = stemmer.stem(word)
    print(f"{word:15}{stem}")
```



```
File Edit Format Run Options Window Help
import nltk
from nltk.stem import PorterStemmer
nltk.download('punkt')
stemmer = PorterStemmer()
words = [
    "running",
    "playing",
    "studies",
    "connected",
    "happiness",
    "cars"
]

print("Original Word\tStemmed Word")
for word in words:
    stem = stemmer.stem(word)
    print(f"{word:15}{stem}")

IDLE Shell 3.14.3
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:
AMD64) on win32
Enter "help" below or click "Help" above for more inform
>>>
===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメン
=====
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\ADMIN\AppData\Roaming\nltk_data
[nltk_data] Package punkt is already up-to-date!
Original Word  Stemmed Word
running        run
playing        play
studies        studi
connected      connect
happiness      happi
cars           car
>>> |
```

Program 6:

Coding

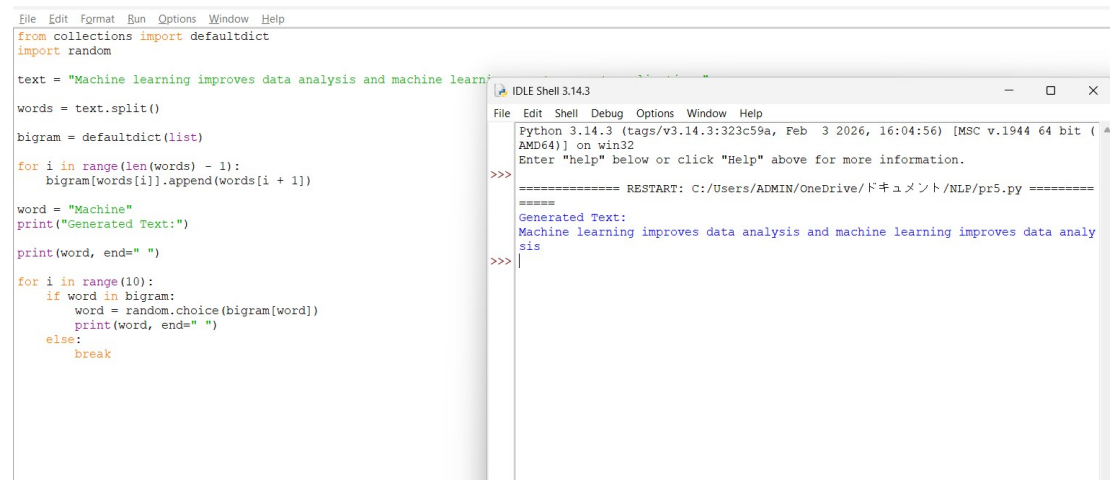
Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.

Coding

```
from collections import defaultdict
import random
```

```
text = "Machine learning improves data analysis and machine learning creates smart
applications"
words = text.split()
```

```
bigram = defaultdict(list)
for i in range(len(words) - 1):
    bigram[words[i]].append(words[i + 1])
word = "Machine"
print("Generated Text:")
print(word, end=" ")
for i in range(10):
    if word in bigram:
        word = random.choice(bigram[word])
        print(word, end=" ")
    else:
        break
```



The image shows a Python script in an IDE window on the left and its execution output in a terminal window on the right. The script implements a bigram model to generate text from a seed word 'Machine'. The terminal output shows the generated text: 'Machine learning improves data analysis and machine learning improves data analysis'.

```
File Edit Format Run Options Window Help
from collections import defaultdict
import random

text = "Machine learning improves data analysis and machine learning improves data analysis"
words = text.split()

bigram = defaultdict(list)

for i in range(len(words) - 1):
    bigram[words[i]].append(words[i + 1])

word = "Machine"
print("Generated Text:")
print(word, end=" ")

for i in range(10):
    if word in bigram:
        word = random.choice(bigram[word])
        print(word, end=" ")
    else:
        break
```

```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/pr5.py =====
>>>
Generated Text:
Machine learning improves data analysis and machine learning improves data analysis
>>>
```